Document 03: Financial DRG Algorithms

CodePerfect Auditor — Revenue Cycle Management Engine

Project: CodePerfect Auditor | **Version:** 1.0 | **Date:** 31-03-2026 **Submitted To:** Virtusa Hackathon | **Institution:** Saveetha Engineering College

---

Overview

The financial engine of CodePerfect Auditor is the most commercially significant component of the platform. It transforms raw AI-generated ICD codes into a precise **Diagnosis Related Group (DRG) financial estimate** — telling the hospital exactly how much they can legally bill a payer for the patient visit.

The system operates in two sequential phases:

1. **ICD Code Scoring & Selection** (Node 6 — `icd_decision.py`): Determines the optimal primary, secondary, and additional diagnosis codes with their financial weights.
2. **CPT/DRG Revenue Calculation** (Node 9 — `financial_calculator.py`): Applies hospital-specific multipliers to CPT procedure codes to compute the gross charge.

Part 1: ICD Financial Weight Scoring — `agents/icd_decision.py`

Why Not Use an LLM for Financial Decisions?

Healthcare auditors and CMS regulations require **deterministic, auditable** coding decisions. An LLM's probabilistic output cannot be legally justified to a payer who challenges a denied claim. The ICD Decision Engine therefore uses a **pure mathematical scoring algorithm** with transparent, documented weights.

Step 1: MCC and CC Flag Detection from the Database

Before scoring begins, each candidate code is loaded from the database with its clinical and financial metadata:

```
# From the snomed_icd_map query result — these fields come from icd_codes table
candidate = {
    "code":               "E11.22",
    "description":        "Type 2 diabetes mellitus with diabetic chronic kidney
disease",
    "is_billable":        True,    # Only billable (leaf) codes generate
reimbursement
    "is_cc":              False,   # Complication/Comorbidity — adds CC_ADJUSTMENT
to DRG
    "is_mcc":             True,    # Major CC — adds MCC_ADJUSTMENT (higher
reimbursement)
    "base_reimbursement": 5000.0,  # CMS baseline DRG rate for this code in USD
    "confidence":         0.87,    # Similarity score from SNOMED crosswalk or
embedding
    "mapping_type":       "exact"  # Trustworthiness of the ontology path
}
```

Explanation

The `is_mcc` and `is_cc` flags map directly to the CMS DRG weight system. When a patient has a **Major Complication/Comorbidity (MCC)**, the hospital DRG weight increases significantly — meaning the payer must legally reimburse a higher amount. If the AI detects an MCC code in Node 6 that a human coder missed, the revenue uplift can be tens of thousands of dollars per case.

Step 2: Non-Billable Code Filter

```
# Filter: remove any candidate that is not a billable leaf-level code
# ICD has many "header" codes (e.g. "E11") that cannot appear on a claim form
candidates = [c for c in candidates if c.get("is_billable", True)]
```

Explanation

ICD-10 codes are hierarchical. Category codes like `E11` ("Type 2 diabetes mellitus") are headers that organize the classification, but they are NOT billable on an insurance claim. Only leaf-level specificity codes like `E11.22` or `E11.3211` are accepted by payer clearinghouses. Sending a non-billable code results in automatic claim rejection.

Step 3: The Weighted Composite Scoring Formula

This is the core financial algorithm. Every candidate code receives a score from 0.0 to 1.0.

```
def _final_score(candidate: dict, entities: dict, raw_text: str = "") -> float:
    """
    Weighted composite scoring formula for ICD code selection:
      - Confidence (40%):   Raw similarity score from the ontology or embedding
model
      - Specificity (30%):  How clinically detailed is this code? (longer = better)
      - Consistency (20%):  Do the code's terms appear in the clinical evidence
text?
      - Combination (10%):  Does this code cover multiple conditions? (preferred by
ICD guidelines)
      - Negation Penalty:   Heavy deduction if chart text contradicts the code's
implications
    """
    confidence  = float(candidate.get("confidence", 0.85))
    specificity = _specificity_score(candidate, entities)
    consistency = _clinical_consistency_score(candidate, entities)
    combination = _combination_code_priority(candidate)
    negation    = _negation_penalty(candidate, entities, raw_text)

    score = (
        confidence  * 0.40 +
        specificity * 0.30 +
        consistency * 0.20 +
        combination * 0.10 +
        negation            # Negative value — acts as a penalty
    )
    return round(max(0.0, min(score, 1.0)), 4)
```

Step 4: Specificity Scoring — Financial Code Granularity

```
COMPLICATION_KEYWORDS = [
    r"\bwith\b",     # Word boundary to avoid matching "without"
    "complicated by", "chronic kidney", "acute", "stage",
    "neuropathy", "retinopathy", "nephropathy", "failure",
]

def _specificity_score(candidate: dict, entities: dict) -> float:
    """
```

```
    Score the clinical and financial specificity of a candidate code.
    Longer, more qualified codes capture higher DRG weights.
    """
    code        = candidate.get("code", "")
    description = candidate.get("description", "").lower()

    # Longer codes = more specific = higher reimbursement potential
    # E11.22 (length 5) > E11.2 (length 4) > E11 (length 3)
    score = len(code) * 0.15

    # If the code description and clinical text both mention a complication,
    # boost the score — this justifies the higher-value code selection
    diag_text = " ".join(d.get("text", "").lower() for d in
entities.get("diagnoses", []))
    for kw in COMPLICATION_KEYWORDS:
        if _kw_match(description, [kw]) and _kw_match(diag_text, [kw]):
            score += 0.2

    return round(min(score, 1.0), 4)
```

Explanation

Code length correlates strongly with financial granularity in ICD-10. A 3-character code (`E11`) is a category header and unbillable. A 7-character code (`E11.3211`) captures "Type 2 diabetes mellitus with unspecified diabetic retinopathy with macular edema" — a highly specific condition with a much higher DRG weight than the base diabetes code. The algorithm specifically rewards codes that capture complications present in the clinical text, which directly translates to higher legally defensible reimbursement.

Step 5: Negation Penalty — Preventing Overcoding Fraud

```
NEGATION_PHRASES = [
    "no complications", "without complications", "no evidence of",
    "no kidney disease", "no renal", "no neuropathy", "no retinopathy",
    "normal kidney function", "kidney function normal",
]

def _negation_penalty(candidate: dict, entities: dict, raw_text: str = "") ->
float:
    """
    Apply a financial penalty if a code implies complications that the chart
    explicitly negates. This prevents fraudulent overcoding.
    """
    description = candidate.get("description", "").lower()
```

```
    # Only apply penalty to codes that imply complications
    is_complication_code = _kw_match(description, [
        r"\bwith\b", "complicated by", "chronic kidney", "neuropathy", "failure"
    ])
    if not is_complication_code:
        return 0.0

    # Search both the LLM-extracted entities AND the full raw text for negation
language
    entity_text  = " ".join(
        (d.get("text", "") + " " + d.get("evidence_text", "")).lower()
        for d in entities.get("diagnoses", [])
    )
    combined_text = (entity_text + " " + raw_text.lower()).strip()

    for phrase in NEGATION_PHRASES:
        if phrase in combined_text:
            return -0.4   # Significant penalty — this code is NOT supported by the
chart

    return 0.0
```

Explanation

Overcoding — billing for conditions that aren't clearly documented — is a federal healthcare fraud offense under the False Claims Act, with penalties up to $25,000 per claim. The negation penalty system scans both the LLM-extracted entities and the full raw chart text for explicit negation language. If a coder selects a complication code like `E11.22` (diabetic kidney disease) but the chart says "no renal complications", this code receives a -0.4 score penalty, ensuring the safer `E11.9` code wins instead.

Step 6: Gold Standard Override — Direct Keyword Mapping

```
GOLD_STANDARD_KEYWORDS = {
    "nstemi":                           "I21.4",  # Non-ST Elevation MI
    "non-st elevation":                 "I21.4",
    "stemi":                            "I21.3",  # ST Elevation MI
    "acute systolic heart failure":     "I50.21",
    "acute on chronic systolic heart failure": "I50.23",
}

def _apply_gold_standard_keywords(candidates: list, raw_text: str) -> list:
    """
```

```
    For high-stakes cardiac and critical care diagnoses, specific medical
    abbreviations trigger a near-certain (0.98) confidence override.
    This ensures correct coding for the most financially significant conditions.
    """
    for candidate in candidates:
        for keyword, exact_code in GOLD_STANDARD_KEYWORDS.items():
            if keyword in raw_text.lower():
                if candidate.get("code") == exact_code:
                    candidate["final_score"] = 0.98  # Override all other scoring
    return sorted(candidates, key=lambda x: x["final_score"], reverse=True)
```

Explanation

NSTEMI (`I21.4`) and STEMI (`I21.3`) are among the highest-DRG-weight cardiac conditions in the CMS payment schedule. If the clinical chart contains these specific abbreviations, the algorithm overrides the composite scoring formula and directly assigns a 0.98 confidence to the correct code. This prevents the scoring formula from accidentally selecting a generic "chest pain" code (`R07.9`) when the chart clearly documents a heart attack — a mistake that could cost the hospital $15,000+ in lost reimbursement per case.

Step 7: Multi-Code Output — Primary, Secondary, Additional

```
# Build a ranked multi-code list following CMS billing hierarchy
icd_codes = [{
    "code":               winner["code"],
    "description":        winner.get("description", ""),
    "role":               "primary",           # The main billing diagnosis
    "final_score":        winner["final_score"],
    "is_mcc":             winner.get("is_mcc", False),
    "is_cc":              winner.get("is_cc", False),
    "base_reimbursement": winner.get("base_reimbursement", 0),
    "rationale":          _rationale(winner, "primary"),
}]

# Include runner-up codes above the 0.40 minimum confidence threshold
role_labels = ["secondary", "additional"]
role_idx = 0
for c in scored[1:]:
    if c["final_score"] >= 0.40 and role_idx < len(role_labels):
        icd_codes.append({
            "code":    c["code"],
            "role":    role_labels[role_idx],  # "secondary" or "additional"
```

```
            "is_mcc":  c.get("is_mcc", False),
            "rationale": _rationale(c, role_labels[role_idx]),
        })
        role_idx += 1
```

Explanation

CMS billing allows — and in many cases requires — multiple ICD codes per claim. Secondary and additional codes capture **comorbidities** (pre-existing conditions that affect treatment) and **additional specificity codes** (providing more detail about the primary diagnosis). Including a valid secondary CC code can shift a patient from DRG tier 1 to DRG tier 2, increasing reimbursement by $2,000–8,000 per admission. The 0.40 minimum threshold ensures only well-supported codes are included — preventing speculative coding that could trigger a payer audit.

Part 2: CPT/DRG Revenue Calculation — `agents/financial_calculator.py`

How Hospital Revenue is Calculated

```
@safe_node("financial_calc")
async def financial_calculator_node(state: CodingState) -> CodingState:
    """
    Applies the hospital-specific pricing multiplier to CPT procedure codes
    and computes the total estimated gross revenue for the patient encounter.
    """
    cpt_codes = state.get("cpt_codes", [])

    # FALLBACK: If no CPT codes were resolved (Node 3 found nothing),
    # fall back to ICD base_reimbursement values from the database.
    # This ensures every coded case always produces a non-zero financial estimate.
    if not cpt_codes:
        icd_codes = state.get("icd_codes", [])
        icd_total = round(sum(float(c.get("base_reimbursement", 0)) for c in
icd_codes), 2)
        state["financial_summary"] = {
            "total_estimated_revenue": icd_total,
            "pricing_multiplier": 1.0,   # No org multiplier applied in fallback
            "line_items": []
        }
        return state
```

```
    # Retrieve the hospital-specific pricing multiplier from org_settings
    org_id     = state.get("org_id")
    multiplier = _get_org_multiplier(supabase, org_id)  # Defaults to 1.0 on
failure

    # Apply the multiplier to every CPT code — this simulates the hospital's
    # gross charge, which differs from the CMS national benchmark rate
    line_items = []
    total = 0.0

    for cpt in cpt_codes:
        base_price   = float(cpt.get("base_price", 0.0))  # National CMS rate
        gross_charge = round(base_price * multiplier, 2)  # Hospital-specific
charge
        total       += gross_charge

        line_items.append({
            "code":          cpt.get("code"),         # e.g. "93306"
            "description":   cpt.get("description"),  # "Echocardiography,
complete"
            "base_price":    base_price,              # e.g. $1,200.00
            "multiplier":    multiplier,              # e.g. 1.5x for private
hospital
            "gross_charge":  gross_charge,            # e.g. $1,800.00
            "confidence":    cpt.get("confidence"),   # Semantic similarity score
        })

    state["financial_summary"] = {
        "total_estimated_revenue": round(total, 2),
        "pricing_multiplier":      multiplier,
        "line_items":              line_items
    }
    return state
```

Explanation — How the Multiplier Works

The `cpt_pricing_multiplier` in `org_settings` is the single most powerful financial control in CodePerfect's architecture. Here is a real-world example:

Scenario	CPT Code	CMS Base Rate	Multiplier	Hospital Gross Charge
Government Hospital	99233	$850.00	1.0x	$850.00
Private Hospital A	99233	$850.00	1.5x	$1,275.00
Premium Hospital B	99233	$850.00	2.2x	$1,870.00

The multiplier reflects the hospital's contracted rate with private insurers. A multiplier of `1.5x` means the hospital has negotiated a contract paying 50% above CMS Medicare base rates. CodePerfect reads this from the database on every coding session, ensuring financial estimates are accurate for each individual hospital tenant.

The Multiplier Fetch Function

```python
def _get_org_multiplier(supabase: Client, org_id: str) -> float:
    """
    Fetch the cpt_pricing_multiplier for a given org from org_settings.
    Falls back gracefully to DEFAULT_MULTIPLIER (1.0) on any database failure.
    The 1.0 default represents the national CMS average — never inflated.
    """
    try:
        response = (
            supabase.table("org_settings")
            .select("cpt_pricing_multiplier")
            .eq("organization_id", org_id)
            .limit(1)
            .execute()
        )
        if response.data and len(response.data) > 0:
            return float(response.data[0]["cpt_pricing_multiplier"])
    except Exception as e:
        log.warning("multiplier_fetch_failed", org_id=org_id, error=str(e))

    return 1.0  # Safe fallback — never inflates revenue on failure
```

Explanation

The `DEFAULT_MULTIPLIER = 1.0` choice is a deliberate financial safety design. If the database connection fails (network timeout, Supabase outage), the system must not inflate revenue estimates. Always falling back to the national CMS average (1.0x) guarantees that CodePerfect never presents a falsely optimistic financial figure. This is what separates a healthcare-grade tool from a naive prototype.

Part 3: Risk Score — Financial Audit Trigger Calculation

After the financial calculation, Node 8 computes the **Audit Risk Score** — the probability that this coding decision will trigger a payer audit or denial.

```
DISCREPANCY_RISK = {
    "EXACT_MATCH":              0.0,   # AI agrees with human → safe
    "NO_COMPARISON":            0.1,   # No human code to compare → minor
uncertainty
    "SPECIFICITY_IMPROVEMENT":  0.2,   # AI found more specific code → low risk
    "CODE_DIVERGENCE":          0.45,  # AI and human chose different categories →
medium risk
    "OVERCODING":               0.5,   # AI chose higher-value code → payer audit
risk
    "UNSUPPORTED_CODE":         0.6,   # Human's code not found in ontology → high
risk
}

def _compute_risk(state: CodingState) -> tuple[float, str]:
    confidence        = float(state.get("confidence_score", 0.5))
    discrepancy       = state.get("discrepancy_type", "NO_COMPARISON")
    delta             = abs(state.get("financial_delta") or 0.0)

    base_risk         = round(1.0 - confidence, 4)      # Low AI confidence = high
risk
    discrepancy_boost = DISCREPANCY_RISK.get(discrepancy, 0.2)
    delta_boost       = min(delta / 5000.0, 0.2)        # Larger $ gap = higher
audit risk

    # DRG-specific boosts
    drg_flag = state.get("drg_flag")
    if drg_flag == "MCC_MISSED":
        mcc_boost = 0.20    # Missed major comorbidity = significant undercoding
risk
    elif drg_flag in ("CC_MISSED", "MCC_OVERCODED"):
        mcc_boost = 0.15

    # Weighted composite risk formula
    raw_score = (
        base_risk         * 0.4 +
        discrepancy_boost * 0.4 +
        delta_boost       * 0.1 +
        mcc_boost         * 0.1
    )
    score = round(min(raw_score, 1.0), 4)
    label = "LOW" if score < 0.35 else ("MEDIUM" if score <= 0.70 else "HIGH")
    return score, label
```

Explanation — Output on the Dashboard

The risk score directly controls the **Risk Assessment widget** displayed on the CodePerfect hospital dashboard. As seen in the application screenshots:

Risk Score	Label	Dashboard Color	Audit Probability
0.00 – 0.34	LOW	Green	< 15% payer challenge
0.35 – 0.70	MEDIUM	Amber	15–50% payer challenge
0.71 – 1.00	HIGH	Red	> 50% payer challenge

The **financial_delta** component is critical — if the AI's code generates $5,000 more revenue than the human coder's code, that 5,000 gap itself is a red flag for payer auditors, and the algorithm reflects this by boosting the risk score proportionally. This allows the RCM department to prioritize which high-delta cases need a senior coder to manually validate before claim submission.

End-to-End Financial Example

Input: Clinical chart describing Type 2 Diabetes with Stage 3 Chronic Kidney Disease.

Step	Node	Output	Financial Impact
1	Document Processor	raw_text extracted	—
2	Clinical Extractor	`{"diagnoses": [{"text": "T2DM with CKD stage 3"}]}`	—
3	CPT Resolver	CPT `99233` (hospital visit, level 3)	Base: $850
4	SNOMED Resolver	`73211009` (Diabetes mellitus type 2)	—
5	SNOMED→ICD Mapper	Candidate: `E11.22` (T2DM + CKD)	is_mcc: True
6	ICD Decision	Winner: `E11.22`, score: 0.87	$5,000 base DRG
7	Audit Comparison	SPECIFICITY_IMPROVEMENT vs human E11.9	+$3,200 delta
8	Risk Scorer	risk_score: 0.18, label: LOW	—

Step	Node	Output	Financial Impact
9	Financial Calculator	gross_charge: $1,275 (1.5x multiplier)	**Total: $6,275**

Result displayed on dashboard: `$6,275 estimated reimbursement | 87% AI Confidence | 18% Audit Risk`

CodePerfect Auditor | Virtusa Hackathon 2026 | Saveetha Engineering College